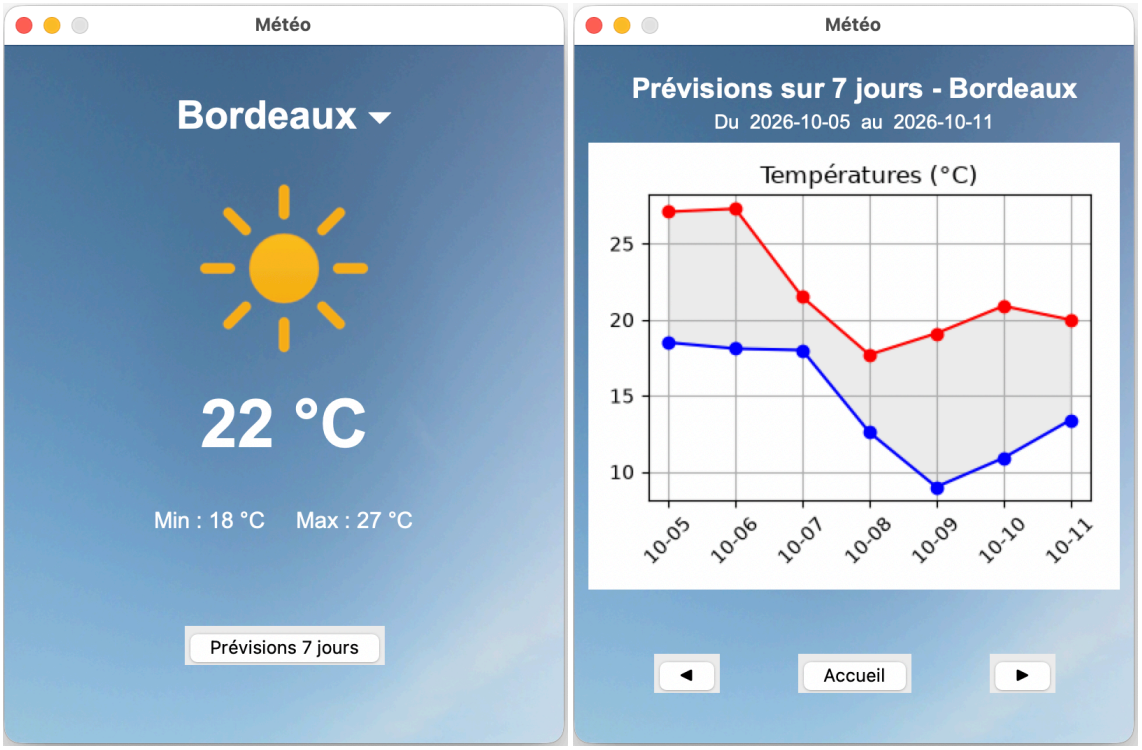
XTI301 Ecosystème Python 2026 - 2027
Projet de fin de module

Le projet peut être réalisé par groupes de 2 ou 3.
Le projet est à rendre au cours de votre dernière séance, sur la page Moodle prévue à cet effet.

Le code n'est pas évalué. Vous êtes donc incités à utiliser le moins possible l'intelligence artificielle pour réaliser ce projet. Votre compréhension du projet sera évaluée par des questions en lien avec le projet dans le devoir final du module, qui aura lieu à la dernière séance.

Objectif du projet

L'objectif de ce TP est de développer une **application météo**, qui affiche la météo du jour et les prévisions sur les prochains jours. L'application pourra avoir cet aspect :



L'application comprend deux écrans :

- **L'écran d'accueil** contient le nom d'une ville, l'icône du temps qu'il fait, la température actuelle et les températures maximales et minimales du jour. L'écran contient également un bouton pour passer au deuxième écran.
- **L'écran des prévisions** contient un graphique Matplotlib des températures minimales et maximales sur 7 jours.

Pour réaliser ce projet, nous utiliserons :

- La librairie **Tkinter**, permettant de créer des interfaces graphiques, *GUI* (*Graphical User Interfaces*).
- Le site web Open-meteo.com qui fournit des données météo quotidiennes gratuitement, importables via son API.
- La librairie Matplotlib pour afficher les prévisions sous forme de graphique.

1. Prise en main de Tkinter

1. Dans le fichier `main.py`, danas la fonction `main`, créez une fenêtre Tkinter.

   ```
   global window, canvas
   window = tk.Tk()
   ```

   Rendez cette fenêtre **non-redimensionnable** (*resizable*). Donnez-lui un titre.

2. Créez un Canvas, attaché à la fenêtre `window`, en spécifiant sa largeur et sa hauteur.

   ```
   canvas = tk.Canvas(window, width = WIDTH, height = HEIGHT)
   canvas.pack()
   ```

3. Appelez les fonctions suivantes :

   ```
   load_background(canvas)
   draw_welcome_screen(window, canvas)
   window.mainloop()
   ```

4. Implémentez la fonction `load_background(canvas)` dans `menu.py`. Cette fonction doit charger dans une variable `image` l'image de fond `"assets/blue_sky.png` et créer une image attachée au `canvas`.

   ```
   global image_background
   image = Image.open("assets/blue_sky.jpg").convert("RGBA")
   image = image.resize((WIDTH, HEIGHT))
   image_background = ImageTk.PhotoImage(image)
   ```

   Grâce à la fonction `canvas.create_image()` (documentation), créez enfin une image attachée au coin en **haut à gauche** du `canvas`.

2. Affichage des informations du jour

On considèrera les données d'exemple suivantes :

```
temperature = 19.7
code = 0
t_min, t_max = 18.0, 27.4
```

1. Ecrivez la fonction `draw_welcome_screen(window, canvas)` qui affiche l'écran d'accueil, avec :

- Le nom de la ville `city`
- La température actuelle
- Les températures minimales et maximales du jour

```
canvas.create_text(x, y, text = "texte à afficher",
                   font = ..., fill = "white", tags = "ecran")
```

2. Ajoutez à votre `canvas` l'image d'icône correspondant à la météo actuelle : un soleil pour `"sunny"`, un nuage pour `"cloudy"`... Les icônes sont situées dans le dossier `assets/icons`.



3. Ajoutez à votre `canvas` un **bouton** permettant d'appeler la fonction `draw_forecast(windows, canvas)`, qui nous dirige vers le menu des prévisions.

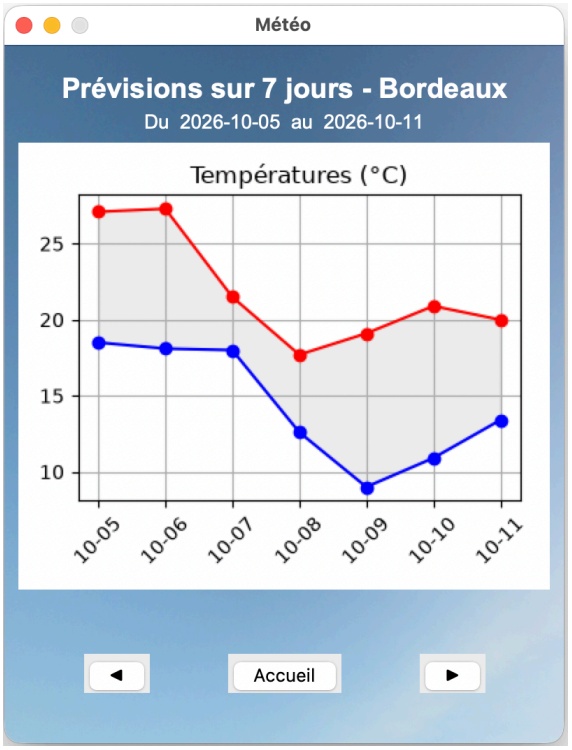
```
button = tk.Button(window, text = "...", command = lambda : """à
compléter ...""")
canvas.create_window(x, y, window = button, tags = "ecran")
```

3. Affichage des prévisions sur 7 jours

On considèrera les données d'exemple suivantes :

```
dates = ['2026-10-06', '2026-10-07', '2026-10-08', '2026-10-09', '2026-10-10', '2026-10-11', '2026-10-12']
temperatures_min = [18.0, 17.8, 12.6, 7.9, 12.3, 13.4, 13.1]
temperatures_max = [27.4, 20.6, 17.6, 18.7, 20.2, 20.0, 20.7]
```

1. Dans le fichier `plot.py`, écrivez la fonction `create_plot(dates, temperatures_min, temperatures_max)`, qui prend en entrée trois listes, contenant des dates, et les températures minimales et maximales sur ces dates. La fonction crée et retourne une `Figure` Matplotlib avec les courbes de températures minimales et maximales sur les dates fournies.



2. Ecrivez la fonction `draw_forecast(window, canvas)`, qui affiche l'écran des prévisions, avec :

- La mention des prévisions sur 7 jours
- La ville actuelle
- La période concernée
- Le graphique obtenu via Matplotlib
- Un bouton de retour à l'accueil

4. Requêtes à l'API d'Open-meteo.com

1. Copiez cet URL dans un navigateur internet, et observez le résultat : https://api.open-meteo.com/v1/forecast?latitude=44.84&longitude=-0.58&current=temperature_2m,weather_code&daily=temperature_2m_max,temperature_2m_min&timezone=auto&forecast_days=1

2. En vous inspirant de la requête et en vous aidant de la documentation de l'API Open-Meteo.com, créez le dictionnaire `parameters` contenant les paramètres à envoyer avec la requête à Open-meteo.com. On souhaite obtenir les informations de votre ville actuelle (Paris ou Bordeaux) pour **la journée d'aujourd'hui** :

   - la **température actuelle**
   - les températures **minimales** et **maximales** du jour
   - le code indiquant la météo actuelle (*Weather code*).

```
parameters = {
    # ...
    # à compléter
    # ...
    "timezone": "auto",
    "forecast_days": 1,
}
response = requests.get(URL_API, params = parameters, timeout = 10)
data = response.json()
```

3. Affichez le dictionnaire `data` obtenu, et renvoyez les quatres valeurs attendues :

```
temperature_now = # data[...][...]
code = # data[...][...]
t_min_today = # data[...][...]
t_max_today = # data[...][...]
return temperature_now, code, t_min_today, t_max_today
```

4. Ecrivez la fonction `get_forecast(city, start_day)`, qui envoie une requête à Open-Meteo.com, afin d'obtenir les **températures minimales** et **maximales** pour chaque jour sur la période **de 7 jours à partir de `start_day`**.

```
end_day = # ...
parameters = {
     # ...
     # à compléter
     # ...
     "start_date": start_day.isoformat(), # pour avoir le format "2025-
01-31"
     "end_date": end_day.isoformat(),
     "timezone": "auto",
}
response = requests.get(URL_API, params = parameters, timeout = 10)
data = response.json()
```

5. Renvoyez les valeurs attendues : les listes des **dates** et des températures minimales et maximales.

```
dates = # data[...][...]
temperatures_min = # data[...][...]
temperatures_max = # data[...][...]
return dates, temperatures_min, temperatures_max
```

6. Modifiez vos fonctions `draw_welcome_screen` et `draw_forecast` afin d'afficher les données correctes dans les menus d'accueil et des prévisions.

   Pour intégrer la `Figure` Matplotlib obtenue :

```
zone_graphique = FigureCanvasTkAgg(figure, master = window)
canvas.create_window(x, y, window = zone_graphique.get_tk_widget(),
                     tags = "ecran")
```

5. Fonctionnalités supplémentaires

Améliorez à présent votre application en lui apportant des fonctionnalités supplémentaires.

Quelques exemples :

- Ajouter des boutons "flèches gauche et droite" pour **reculer** ou **avancer** de 7 jours dans le menu des prévisions
- Charger une image de fond d'écran alternative en fonction de l'heure de la journée (on dispose du fichier `assets/night_sky.jpg`)
- Afficher des **icônes de nuit** lorsque celles-ci montrent le soleil en pleine nuit
- Permettre de **changer de ville** via un menu déroulant dans le menu principal.
- Ajouter des icônes représentant la météo de chacun des 7 jours du menu des prévisions.

